Exercise Sheet 01

Introduction to PyTorch and Convolution
This notebook contains one code cell per subtask. Complete the `# TODO` sections and keep your outputs visible before exporting the notebook as PDF.

General Setup

Run the next cell first. It loads the libraries and helper functions that are allowed for the sheet.

Expected file layout: place `Wombat.jpg` in the same directory as this notebook unless a subtask explicitly asks you to load something else yourself.

In [2]:
```
from pathlib import Path
import random

import matplotlib.pyplot as plt
import numpy as np
import torch
import torchvision.models as models
from PIL import Image

plt.rcParams['figure.figsize'] = (8, 6)
plt.rcParams['image.cmap'] = 'gray'

IMAGE_PATH = Path('Wombat.jpg')
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {device}')

# You may use this function to normalize arrays for display.
def normalize_for_display(array):
    array = np.asarray(array, dtype=np.float32)
    array_min = array.min()
    array_max = array.max()
    return (array - array_min) / (array_max - array_min + 1e-8)

# You may use this function to display images in the notebook.
def show_image(array, title, cmap=None):
    plt.figure(figsize=(6, 5))
    plt.imshow(array, cmap=cmap)
    plt.title(title)
    plt.axis('off')
    plt.show()
```

```
Using device: cuda
```

Task 1: Basic Image Manipulation

Task 1.1

Load `Wombat.jpg` using a library of your choice and convert it to a NumPy array.

In [3]:
```
# TODO: Load Wombat.jpg and convert it to a NumPy array called `img`.
img = Image.open('Wombat.jpg')
img = np.array(img)
```

Task 1.2

Print the image shape, explain its meaning, and visualize the image with the shape in the title.

In [59]:
```
# TODO: Print img.shape, explain the dimensions, and visualize the image.
print(f"{img.shape}\n Dimesnions: (image hight, image width, color chanels)"


show_image(img, "wombat_1200x800_rgb")
```

```
(800, 1200, 3)
 Dimesnions: (image hight, image width, color chanels)
```

wombat_1200x800_rgb

Task 1.3

Extract a crop of random size centered at the image center and plot it.

In [5]:
```
# TODO: Create a random centered crop from `img` and store it in `center_cro
h, w, c = img.shape

crop_hight = np.random.randint(1, h)
crop_width = np.random.randint(1, w)

center_x = w//2
center_y = h//2

crop_start_x = center_x - crop_width // 2
crop_end_x = center_x + crop_width // 2
crop_start_y = center_y - crop_hight // 2
crop_end_y = center_y + crop_hight // 2

croped_img = img[crop_start_y:crop_end_y, crop_start_x:crop_end_x]
show_image(croped_img, f"wombat_croped_{crop_width}x{crop_hight}")
```

wombat_croped_267x739

Task 1.4

Randomly permute the RGB color channels of the crop and plot it.

In [6]:
```
# TODO: Randomly permute the channel order of `center_crop` and store the re
channels = [0, 1, 2]

channels_perm = np.random.permutation(channels)
img_croped_permuted = croped_img[:,:,channels_perm]

show_image(img_croped_permuted, f"wombat_croped color channels: {channels_pe
```

wombat_croped color channels: [1 2 0]

Task 1.5

Convert the original image to grayscale and plot it.

In [7]:
```
# TODO: Convert `img` to grayscale, store the result in `gray_img`, and visu

grey_values = np.array([0.299, 0.587, 0.114])

img_greyscale = np.dot(img,grey_values)
show_image(img_greyscale, "wombat_greyscale")
```

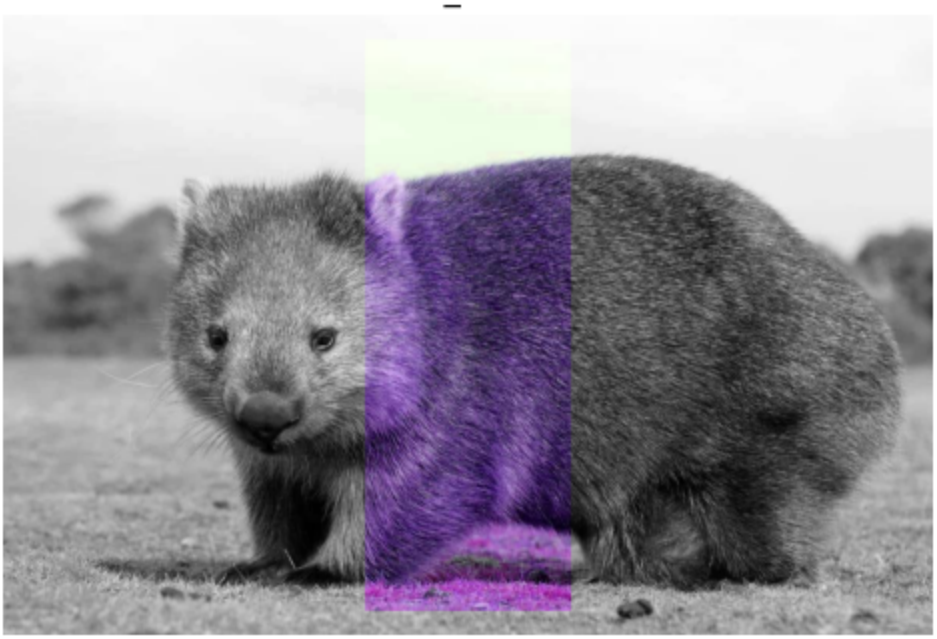
wombat_greyscale

Task 1.6

Insert the color-permuted crop back into the grayscale image and plot the result.

In [8]:
```
# TODO: Insert `permuted_crop` into the grayscale image and store the result
img_combined = np.stack((img_greyscale, img_greyscale, img_greyscale), axis=

img_combined[crop_start_y:crop_end_y, crop_start_x:crop_end_x] = img_croped_
img_combined = img_combined.astype(np.uint8) # convert decimals to integers

show_image(img_combined, "wombat_combined")
```

wombat_combined

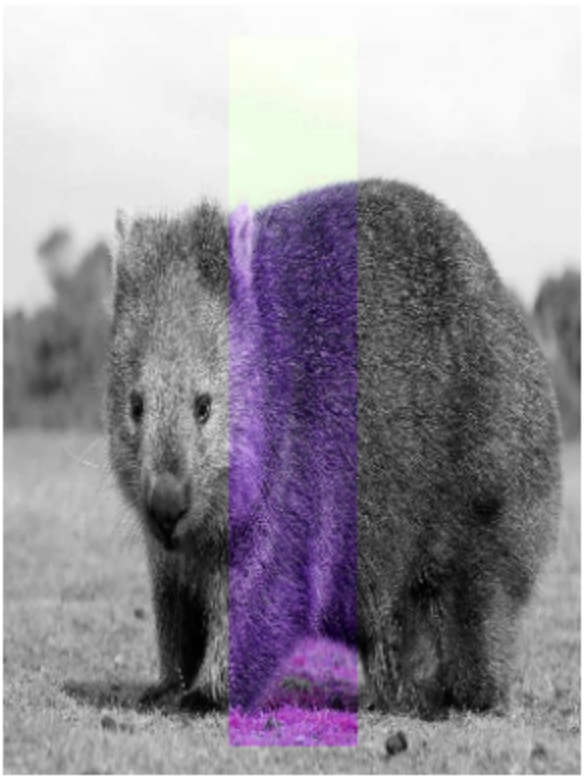


Task 1.7

Resize the grayscale image with inserted color patch by a factor of 1/4 in width and 1/2 in height.

In [9]:
```
# TODO: Resize `gray_with_patch` to the requested output size and visualize

img_combined_downsampled = img_combined[::2, ::4]
show_image(img_combined_downsampled, f"wombat_combined_{img_combined_downsam
```

wombat_combined_300x400



Task 2: Convolution and Filtering

For this task, implement the required convolutions manually with NumPy. Do not call built-in convolution functions.

Task 2.1

Implement single-channel convolution for grayscale images.

In [10]:
```
# TODO: Implement `conv2d_gray`.

def conv2d_gray(image, kernel):
    """Manual 2D convolution / filtering for a single-channel image."""
    h , w = image.shape
    n = kernel.shape[0]
```

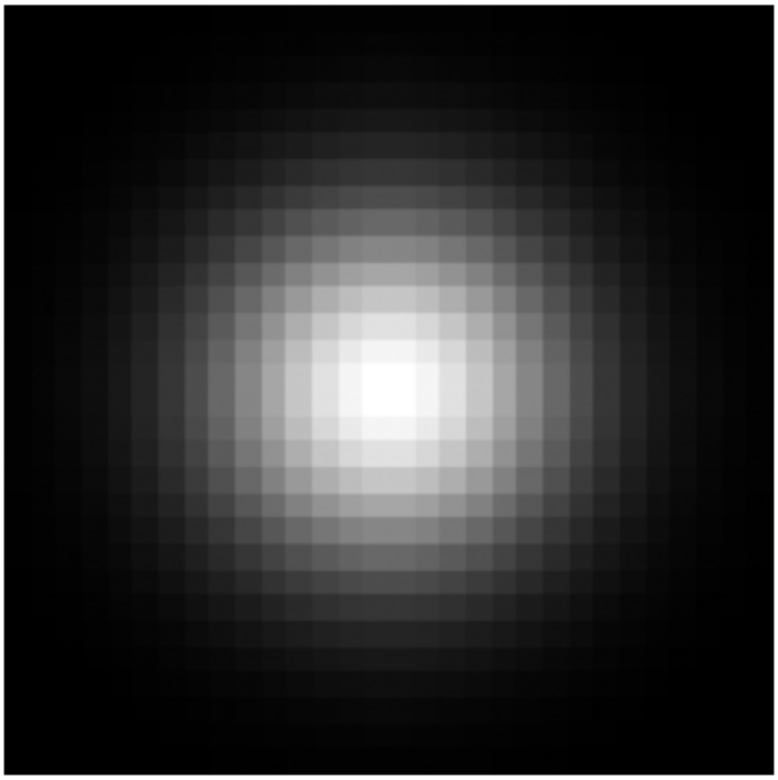
```
    k = n // 2

    # empty output image
    h_max = h - n + 1
    w_max = w -n +1
    output_img = np.zeros((h_max, w_max))

    for y in range(k, h - k):
        for x in range(k, w  - k):
            patch = image[y-k : y+n-k, x-k : x+n-k]
            pixel_value = np.sum(patch*kernel)

            output_img[y-k, x-k] = pixel_value

    return output_img
```

Task 2.2

Implement a Gaussian kernel, visualize it, apply it to the grayscale wombat image, and comment on the effect of `n` and `sigma`.

In [61]:
```
# TODO: Implement `gaussian_kernel`, apply it, visualize the results, and co

def gaussian_kernel(n, sigma):
    """Return a normalized Gaussian kernel of shape (n, n)."""
    k = n // 2

    # create grid
    grid_line = np.linspace(-k, k, n)
    grid_row, grid_column = np.meshgrid(grid_line, grid_line)

    # apply formula
    kernel = np.exp(-(grid_row**2 + grid_column**2) / (2 * sigma**2))

    # normalization
    return kernel / np.sum(kernel)

# create kernel and visualize kernel
n, sigma = 30, 5
kernel = gaussian_kernel(n, sigma)
kernel_display = normalize_for_display(kernel)
kernel_200 = np.array(Image.fromarray(kernel_display, mode='F').resize((200,
show_image(kernel_200, f"kernel_{kernel_200.shape[0]}x{kernel_200.shape[1]}

# apply kernel
img_greyscale_blured = conv2d_gray(img_greyscale, kernel)
show_image(img_greyscale_blured, "title")


comment_task_2_2 = 'Sigma changes the intensity of the blur, meaning how sha
print(comment_task_2_2)
```

kernel_200x200 n=30 sigma=5



title

Sigma changes the intensity of the blur, meaning how sharp or soft the image looks.
n controlls how many neighbouring pixels are taken into account when calculating the blur.
This only becomes visible in the image when n is closer to sigma and the blur gets "cut off"

Task 2.3

Extend the convolution implementation to multi-channel inputs.

In [12]:
```
# TODO: Implement `conv2d_multi`.

def conv2d_multi(image, kernel):
    """Manual convolution for multi-channel input and a single output channe
    h, w, c = image.shape
    n = kernel.shape[0]
    k = n // 2

    # empty output image
    h_max = h-n+1
    w_max = w-n+1
    output_img = np.zeros((h_max, w_max))

    for y in range(k, h-k):
        for x in range(k, w-k):
            patch_3d = image[y-k:y+k+1, x-k:x+k+1, :]
            pixel_value = np.sum(patch_3d*kernel)

            output_img[y-k, x-k] = pixel_value

    return output_img
```

Task 2.4

Use a 1x1 kernel to convert an RGB image to grayscale, apply it to the original wombat image, and explain the role of the kernel weights.

In [13]:
```
# TODO: Build a 1x1x3 kernel, apply it with `conv2d_multi`, and explain the

kernel_1x1 = np.array([[[0.299, 0.587, 0.114]]])


img_blured = conv2d_multi(img, kernel_1x1)
show_image(img_blured, "wombat_greyscale_conv2d_multi")

answer_task_2_4 = 'The kernel size is only one pixel so there are no neighbo
print(answer_task_2_4)
```

wombat_greyscale_conv2d_multi

```
The kernel size is only one pixel so there are no neighbours to take into ac
count. The weights simply specify
how much each color value (r,g,b) influences the output pixel.
```

Task 2.5

Discuss limitations of manually designed filters and advantages of learning filters from data.

In [14]:
```
# TODO: Replace the text below with your discussion.

discussion_task_2_5 = """
Even though manual filters are more predictable, they can be very hard to de
They also require manual work of humans for the design process.

Learning filters from data on the other hand can lead to the discovery of co
This also allowes the development of complex filters in the background witho
"""
print(discussion_task_2_5)
```

```
Even though manual filters are more predictable, they can be very hard to de
sign when their purpose becomes more complex.
They also require manual work of humans for the design process.

Learning filters from data on the other hand can lead to the discovery of co
mplex patterns which a human might have overseen.
This also allowes the development of complex filters in the background witho
ut permanent supervision of a human.
```

Task 3: Network Visualization and Interpretation

Task 3.1

Load pretrained AlexNet, print its architecture, identify the first convolutional layer, and explain its weight tensor shape.

In [ ]:
```
# TODO: Load pretrained AlexNet, inspect the first convolutional layer, and
weights = models.AlexNet_Weights.DEFAULT
model_alex = models.alexnet(weights=weights)

print(f"Architecture: \n{model_alex}\n")
print(f"First convolutional layer: \n{model_alex.features[0]}\n")
print(f"Weight tensor shape: \n{model_alex.features[0].weight.shape}")

shape_explanation_task_3_1 = """
Shape explanantion:
64: Number of different filters the model uses to identify specific features
3: Number of color channels
11: Kernel height
11: Kernel width
"""
print(shape_explanation_task_3_1)
```

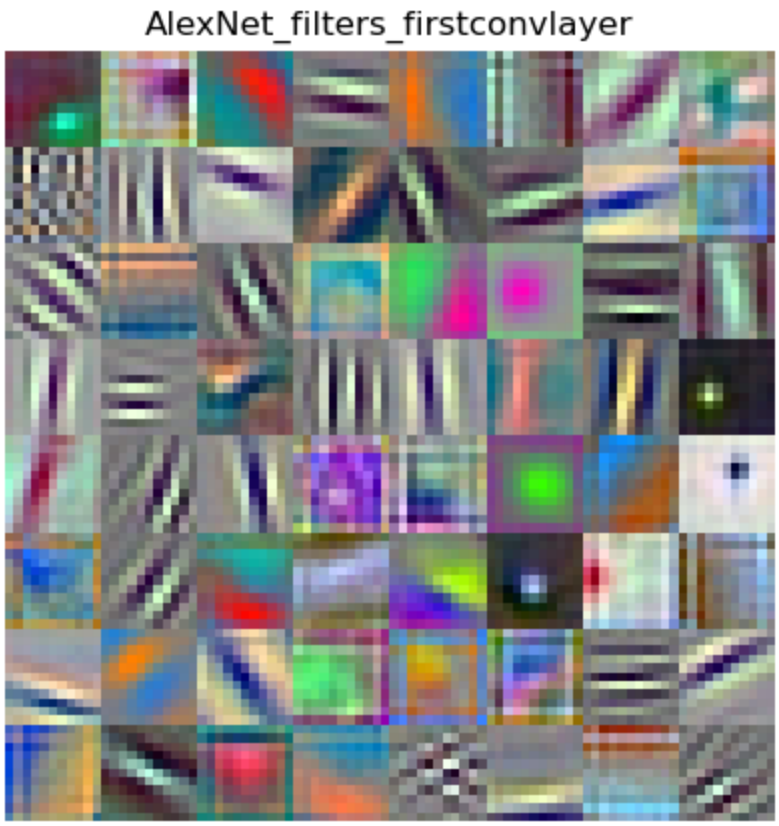
```
Architecture:
AlexNet(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(11, 11), stride=(4, 4), padding=(2, 2))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode
=False)
    (3): Conv2d(64, 192, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode
=False)
    (6): Conv2d(192, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU(inplace=True)
    (8): Conv2d(384, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): ReLU(inplace=True)
    (10): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1))
    (11): ReLU(inplace=True)
    (12): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mod
e=False)
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(6, 6))
  (classifier): Sequential(
    (0): Dropout(p=0.5, inplace=False)
    (1): Linear(in_features=9216, out_features=4096, bias=True)
    (2): ReLU(inplace=True)
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=4096, out_features=4096, bias=True)
    (5): ReLU(inplace=True)
    (6): Linear(in_features=4096, out_features=1000, bias=True)
  )
)

First convolutional layer:
Conv2d(3, 64, kernel_size=(11, 11), stride=(4, 4), padding=(2, 2))

Weight tensor shape:
torch.Size([64, 3, 11, 11])

Shape explanantion:
64: Number of different filters the model uses to identify specific features
in the immage
3: Number of color chanels
11: Kernel height
11: Kernel width
```

Task 3.2

Visualize all learned filters of the first convolutional layer.

In [33]:
```
# TODO: Extract and visualize the filters of the first convolutional layer.
kernel_list = model_alex.features[0].weight.data.cpu().numpy()

# Create grid for kernels
```

```
grid_size = 8
kernel_h, kernel_w = 11, 11
grid = np.zeros((grid_size*kernel_h, grid_size*kernel_w, 3))

for i in range(64):
    cur_kernel = kernel_list[i]
    cur_kernel = cur_kernel.transpose(1,2,0)

    cur_kernel_normalized = normalize_for_display(cur_kernel)

    #calculate grid position
    row = i // grid_size
    column = i % grid_size
    grid[row*kernel_h:(row+1)*kernel_h, column*kernel_w:(column+1)*kernel_w,

show_image(grid, "AlexNet_filters_firstconvlayer")
```



Task 3.3

Discuss what you observe in the first-layer filters.

In [34]:
```
# TODO: Replace the text below with your discussion.

discussion_task_3_3 = """
There are several patterns in the filters:
- color contrasts: several filters visualize the transition from one color t
show points of color on another background, for example green on purple (3,6
- oriented edges: There are several destinct edges in horizontal adn vertica
- intensity transitions: There are several fades between two colors, for exa
```

```
- These primitive objects can be used to "build"/classify more complex objec
"""
print(discussion_task_3_3)
```

```
There are several patterns in the filters:
- color contrasts: several filters visualize the transition from one color t
o another, for example orange to blue (1,5). Others
show points of color on another background, for example green on purple (3,
6)
- oriented edges: There are several destinct edges in horizontal adn vertica
l positions as well as several 45 degree angles
- intensity transitions: There are several fades between two colors, for exa
mple blue to yellow (8,1)
- These primitive objects can be used to "build"/classify more complex objec
ts, like animals or machines
```

Task 3.4-3.7

Compute a saliency map for the predicted class of `Wombat.jpg`.

Work through the following subtasks:

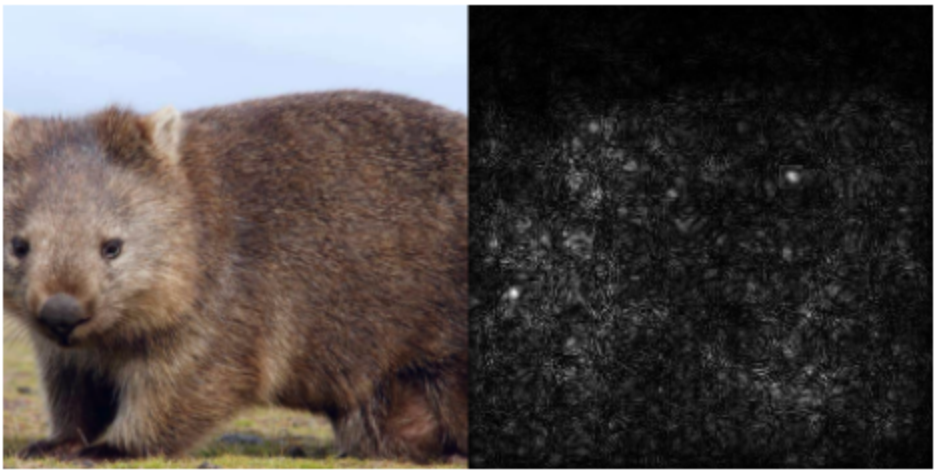
1. Determine the predicted class of the image.
2. Compute the gradient of the predicted class score with respect to the input image.
3. Construct and visualize the saliency map.
4. Briefly explain what high saliency values mean in this context.

In [62]:
```
# TODO 3.4: Load `Wombat.jpg`, preprocess it with the pretrained AlexNet tra
# and determine the predicted class.
test_image = Image.open('Wombat.jpg')

# preprocess
preprocess = weights.transforms()
input_singel = preprocess(test_image)
input_batch = input_singel.unsqueeze(0)

# generate predicition
model_ouput = model_alex(input_batch)

predicted_class_idx = torch.argmax(model_ouput).item()
predicted_class_label = weights.meta["categories"][predicted_class_idx]

print(f'Predicted class index: {predicted_class_idx}')
print(f'Predicted class label: {predicted_class_label}')

# TODO 3.5: Compute the gradient of the predicted class score with respect t
# Hint: enable gradients for the input, reset gradients with `model.zero_gra
# and call `score.backward()`.

input_batch.requires_grad_()
model_alex.zero_grad()
output_grad = model_alex(input_batch)
```

```
target_score = output_grad[0, predicted_class_idx]
target_score.backward()

gradient_wrt_input = input_batch.grad.data.detach().cpu().numpy()[0]
print(f"Gradient: {gradient_wrt_input}")

# TODO 3.6: Construct the saliency map by taking the absolute gradient and c
# the color channels into one 2D map (for example via a maximum over channel
# Visualize the input image and the saliency map side by side.

#absolute gradient values
gradients_abs = np.abs(gradient_wrt_input)

saliency_map = np.max(gradients_abs, axis=0)

# combi fiew
saliency_map_norm = normalize_for_display(saliency_map)
saliency_map_3d_norm = np.stack([saliency_map_norm]*3, axis=-1)
image_np = input_singel.permute(1,2,0).detach().cpu().numpy()
image_np_norm = normalize_for_display(image_np)

combi_fiew = np.concatenate([image_np_norm, saliency_map_3d_norm], axis=1)
show_image(combi_fiew, "saliency_map")

# TODO 3.7: Replace the text below with your discussion.

answer_task_3_7 = """
High saliency values mean that the corresponding place in the image had a hi
Saliency maps can show us what parts of the image were most influential on t
unimportant parts have a high influence.
"""
print(answer_task_3_7)
```

```
Predicted class index: 106
Predicted class label: wombat
Gradient: [[[-2.4634765e-03 -3.2648297e-03 -4.1699503e-03 ... -6.6676596e-04
   -2.3213851e-04  1.6388826e-03]
  [-1.2238831e-03 -1.1523328e-03 -3.9513079e-03 ... -2.9659260e-04
    2.3882295e-04  1.6342485e-03]
  [ 3.5762009e-03  3.1508636e-03 -1.4553007e-03 ...  1.0191872e-03
    4.6917307e-04  2.6612852e-03]
  ...
  [-6.1392540e-04 -1.4380972e-03 -1.0670046e-02 ...  7.2459201e-04
   -8.6667387e-05  7.8102843e-05]
  [ 1.2773499e-03  1.9352902e-03 -3.7849196e-03 ...  1.1165604e-04
   -1.7175279e-04 -1.2422913e-04]
  [-8.5170218e-04  2.0800687e-03 -3.4413540e-03 ... -1.6550928e-04
   -5.3443777e-04 -7.2900404e-04]]

 [[-1.8648924e-03 -3.2453281e-03 -5.3935372e-03 ... -2.6699115e-04
    7.8692858e-05  2.4635107e-03]
  [ 2.8777780e-04  1.2104307e-04 -3.2267533e-03 ... -2.0687307e-04
    9.3842158e-05  2.3243928e-03]
  [ 5.1998287e-03  5.2153775e-03  1.5371541e-03 ...  9.5508678e-04
    5.0950143e-04  3.5920199e-03]
  ...
  [ 7.0448586e-05  6.6974154e-03  7.5070621e-03 ...  7.7837222e-04
    4.7343079e-04  6.1876024e-04]
  [-2.2987924e-03  3.7268824e-03  4.8320629e-03 ...  9.8800811e-05
    6.7515124e-05  1.9097490e-04]
  [-3.5131273e-03 -6.8558700e-04 -2.7688264e-03 ... -1.5836560e-04
   -5.8615959e-04 -4.8512238e-04]]

 [[ 1.7601075e-03  5.9469923e-04 -1.5526385e-03 ...  6.6298235e-04
    1.0380413e-03  3.3172567e-03]
  [ 2.2050343e-03  2.1128962e-03 -1.8225880e-03 ...  3.2500393e-04
    6.5706443e-04  3.0226179e-03]
  [ 5.1614451e-03  5.1357178e-03  5.4154807e-04 ...  8.7932567e-04
    5.6011957e-04  3.3953476e-03]
  ...
  [-1.0060129e-03 -6.3040352e-05 -1.0381550e-02 ...  4.6176914e-04
   -1.0924303e-04 -2.8327623e-04]
  [-1.4190690e-03  4.2008262e-04 -3.3048859e-03 ... -8.7290560e-04
   -1.0832915e-03 -9.4898802e-04]
  [-1.4214347e-03  7.5735903e-04 -2.6489508e-03 ... -2.1277142e-03
   -2.2255785e-03 -2.5241161e-03]]]
```

saliency_map



```
High saliency values mean that the corresponding place in the image had a hi
gh influence on the models prediction.
Saliency maps can show us what parts of the image were most influential on t
he final decision and whether any
unimportant parts have a high influence.
```

Submission Reminder

Before submission, make sure that:

- all `# TODO` sections are completed,
- all requested plots are visible in the notebook output,
- the notebook is exported as PDF,
- the `.ipynb` and PDF are included in the ZIP file following the naming convention from the sheet.